

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..46

LOAD TESTING

 Copy as Markdown

Load testing

You need to know what happens when many clients call your backend at once: whether latency climbs, whether the database pool saturates, and whether errors appear before you ship. Serverpod does not ship a load-testing tool, so you drive the same HTTP API your generated client uses (with Locust, or with that client itself) against a production-like server, then read Insights, health probes, session logs, and Postgres.

The [test tools](#) are a different job. They boot an in-process server in `test` mode and roll the database back after each test, which is how you check correctness, not throughput.

Before you start

- A Serverpod project with the [endpoints](#) you want to measure, and `serverpod generate` already run.
- [Locust installed](#).
- A PostgreSQL instance you run yourself. The [embedded PostgreSQL](#) that `serverpod start` manages is for local development and tests, not load.

Run a production-like server

A run under `serverpod start` logs every session, uses JIT, and hot reloads. Start the same kind of process you would deploy: [production \(or staging\) run mode](#), the project's Dockerfile (it builds with `dart build cli`), and a real Postgres with no `dataPath`.

```
dart run bin/main.dart --mode production
```

That command is enough to match production logging and config. It is still JIT. For numbers you want in production, run the compiled binary from your Dockerfile instead. Leave `--logging` at `normal`



Each Serverpod process runs your Dart code on a single isolate, which is one thread of execution. Synchronous CPU work on that isolate delays every other request on the same process. To use more cores, run more server processes (or pods), not more threads inside one process.

The production config template leaves Redis off. Turn it on if the scenario uses the [global cache](#) or messaging across instances.

Call an endpoint over HTTP

The API server listens on port `8080` by default. Endpoint methods are `POST` requests with a JSON body and `Content-Type: application/json`. An `Authorization` header is optional, and is how you exercise authenticated methods.

The generated client posts to `/endpoint/method` with a JSON object of named arguments:

```
curl -X POST http://localhost:8080/example/hello \  
-H 'Content-Type: application/json' \  
-d '{"name": "World"}'
```

A method on a module uses `{module}.{endpoint}/{method}`. [Streaming methods](#) use WebSockets, not HTTP POST, so they need a separate scenario.

Write a Locust scenario

Locust is the default path: it already speaks this POST-and-JSON shape, and a [community benchmark](#) uses it against Serverpod. Point each task at one method. Give database reads, database writes, and static methods their own files so you can tell them apart.

```
from locust import FastHttpUser, task  
  
class ExampleUser(FastHttpUser):  
    @task  
    def hello(self):  
        self.client.post("/example/hello", json={"name": "World"})
```

Run it against the API server:

```
locust -f locustfile.py --host http://localhost:8080
```

Locust opens a web UI (port `8089` by default, separate from Serverpod's `8080`/`8081`/`8082`). Start with a small user count and raise it until something saturates. For think time between calls, set `wait_time` on the

user class; omit it when you want a raw request ceiling. See [writing a locustfile](#) for spawning, headless runs, and percentiles.

Generic HTTP tools such as k6 and wrk work for the same POST-and-JSON calls. You encode the argument map yourself, and you lose the typed client.

Measure the server

Locust reports client-side requests per second, latency percentiles (p50, p95, p99), error rate, and timeouts. That is only half the picture. Watch the server at the same time.

Health probes: `/livez` (process alive), `/readyz` (database, Redis, and custom indicators), `/startupz` (boot complete). Every server answers them; probe the API port (`8080` by default), since that is the server under load. During a run, `/readyz` is the one that tells you the server should still receive traffic. See [Health checks](#).

Health metrics in [Insights](#): CPU load average, memory, database `SELECT 1` latency, and HTTP connection counts (`active`, `closing`, `idle`). Collection runs once a minute. Leave that interval alone; a shorter one writes extra database rows and can distort the run. CPU and memory metrics are not collected on Windows.

Session and query logs in [Insights](#): endpoint, method, duration, query count, and whether the call or query was slow or failed. In `production`, a session is persisted only if it was slower than one second, it failed, or it produced a log or query entry. Fast successful calls leave no row, which is what you want.

⚠ WARNING

Turning on logging for every session or every query during a load test writes a database row per request and becomes the bottleneck. Keep the production defaults, or disable persistent session logs for the run (`sessionLogs.persistentEnabled: false`, or `SERVERPOD_SESSION_PERSISTENT_LOG_ENABLED=false`). If you need request traces, console logs with `consoleLogFormat: json` are cheaper than the log tables.

Postgres: watch connection count against your pool size (`database.maxConnectionCount`, default `10`, env `SERVERPOD_DATABASE_MAX_CONNECTION_COUNT`). Postgres `max_connections` must be at least the pool size times the number of server processes. See [Configure connection pool size](#).

A valid run has a near-zero error rate (or the failures you meant to provoke), `/readyz` returning `200`, and session-log tables that are not growing by one row per request.

Check these settings before you run

Setting	Why it matters
<code>--mode development</code> (the default)	Logs every session and runs with JIT and hot reload.
JIT (<code>dart run</code> / <code>serverpod start</code>)	Understates production throughput. Use the compiled binary from the Dockerfile.
<code>database.maxConnectionCount</code> (default 10)	Saturates first under database load.
<code>sessionLogs.persistentEnabled</code>	A row per request will cap throughput. See the warning above.
<code>--logging verbose</code>	Extra work on every call.
<code>futureCall.concurrencyLimit</code> (default 1)	Only if the scenario schedules future calls .
<code>healthCheckInterval</code> below one minute	Extra metric rows in Postgres.
Embedded Postgres (<code>dataPath</code>)	Not representative of a hosted database.

Purge or cap the log tables between runs (`cleanupInterval`, `retentionPeriod`, `retentionCount` on [Logging](#)). Unbounded log tables distort later measurements.

Drive load with the generated client

Use the generated Dart client when the scenario needs type-safe models, authentication, or [streaming](#). Locust will not speak WebSockets for you.

```
final client = Client('http://localhost:8080/');
final result = await client.example.hello('World');
```

Give each virtual user its own `Client` (or a small pool), drive the generated stubs, and time calls with `Stopwatch`. The client's default timeout is 20 seconds. Spawn many isolates or processes on the client; a single isolate will under-generate load.

Scenario types

Treat these as separate runs. They stress different parts of the stack:

- Static methods with no database
- Create, read, update, and delete against Postgres
- Authenticated methods, using the same `Authorization` header your app sends ([Authentication basics](#))
- Streaming over WebSocket
- [File uploads](#)
- Mixed user behavior (Locust tasks with `wait_time`)

Troubleshooting

Why do the numbers look better than production?

The server was started with `serverpod start` or without `--mode production`. Development logs every session and runs with JIT and hot reload. Rebuild with the project's Dockerfile and start in production or staging mode.

Why is latency high while CPU is idle?

The database pool is full. The default `maxConnectionCount` is 10. Raise it, and confirm Postgres `max_connections` is large enough for every replica.

Why did Postgres become the bottleneck immediately?

Persistent session or query logging is writing a row per request. Keep production log defaults, or set `sessionLogs.persistentEnabled` to `false` for the run.

Why can't a Dart driver generate enough load?

The driver is running in one isolate. Spawn many isolates or processes on the client, and scale the server with more processes as well.

Related

- [Get started with testing](#): correctness tests with `withServerpod`, not a load-test path.
- [Working with endpoints](#): how the generated client calls your server.
- [Insights](#): live health, session logs, and runtime log settings.
- [Logging](#): which sessions are persisted, and how to turn persistent logs off.
- [Health checks](#): `/livez`, `/readyz`, `/startupz`, and collected CPU, memory, and database metrics.
- [Configuration](#): run modes, YAML, and environment variables.
- [Configure connection pool size](#): the pool that saturates first under database load.
- [Community Locust scripts](#): worked examples that split database and static scenarios.

- [Community benchmark discussion](#): an earlier Locust run against Serverpod, not a current baseline.